MINI PROJECT REPORT

ON

BANKER'S ALGORITHM FOR DEADLOCK AVOIDANCE

Submitted in partial fulfilment of the requirements for the subject

OPERATING SYSTEMS

Name: Suyog Chaudhari

Roll No.: 26ET2052002

Branch: B.Tech CSF

College: Alard University

Submitted to: _____

ABSTRACT

The Banker's Algorithm is a deadlock-avoidance algorithm used by an operating system to allocate resources safely among multiple processes. Before granting a resource request, the algorithm checks whether the allocation would leave the system in a safe state. If a safe sequence exists, the request can be granted; otherwise, the request is postponed. This mini project implements the safety algorithm in the C programming language. It accepts the Allocation, Maximum and Available resource data, calculates the Need matrix, and determines whether the system is safe. The project helps demonstrate resource allocation and deadlock avoidance in a simple, understandable way.

1. INTRODUCTION

An operating system manages resources such as memory, printers, files and other devices. Several processes may request the same resources at the same time. If resources are allocated without careful planning, processes may become blocked and a deadlock may occur. Deadlock avoidance checks resource requests before granting them. The Banker's Algorithm, introduced by Edsger W. Dijkstra, is a well-known method that uses each process's maximum resource requirement to decide whether an allocation is safe.

2. PROBLEM STATEMENT

To develop a C program that checks whether a system is in a safe state using the Banker's Algorithm. The program uses the current Allocation matrix, Maximum demand matrix and Available resource vector to calculate the Need matrix and find a safe sequence, if one exists.

3. OBJECTIVES

- To understand deadlock and deadlock avoidance in an operating system.
- To calculate the remaining resource requirement of each process.
- To check whether the system is in a safe state.
- To display a safe sequence when one is available.
- To implement the algorithm using arrays and loops in C.

4. BASIC CONCEPTS

4.1 Deadlock

A deadlock is a situation in which a group of processes waits indefinitely because each process is waiting for a resource held by another process in the group.

4.2 Safe State

A system is in a safe state if there is at least one order of processes in which every process can obtain its remaining resources, finish execution, and release its resources for the next process.

4.3 Data Structures

- Available: number of currently available instances of each resource type.
- Max: maximum number of instances of each resource type that each process may need.
- Allocation: number of resource instances currently allocated to each process.
- Need: remaining resource requirement, calculated as Need = Max – Allocation.

5. ALGORITHM / WORKING

1. Read the number of processes and resource types.
2. Input the Allocation matrix, Maximum matrix and Available vector.
3. Calculate Need[i][j] = Max[i][j] – Allocation[i][j] for every process and resource type.
4. Set Work = Available and mark every process as unfinished.

5. Find an unfinished process whose Need is less than or equal to Work for every resource type.

6. Assume that process finishes, add its allocated resources to Work, and mark it finished.

7. Repeat until all processes finish or no eligible process can be found.

8. If all processes finish, print the safe sequence. Otherwise, report that the system is unsafe.

6. C PROGRAM

```
#include <stdio.h>

int main(void) {
    int n, m;
    int alloc[10][10], max[10][10], need[10][10];
    int avail[10], work[10], finish[10] = {0};
    int safe[10], count = 0;

    printf("Enter number of processes (max 10): ");
    scanf("%d", &n);
    printf("Enter number of resource types (max 10): ");
    scanf("%d", &m);

    if (n < 1 || n > 10 || m < 1 || m > 10) {
        printf("Invalid number of processes or resources.\\n");
        return 1;
    }

    printf("Enter Allocation matrix:\\n");
    for (int i = 0; i < n; i++)
        for (int j = 0; j < m; j++)
            scanf("%d", &alloc[i][j]);

    printf("Enter Maximum matrix:\\n");
    for (int i = 0; i < n; i++)
        for (int j = 0; j < m; j++) {
            scanf("%d", &max[i][j]);
            if (max[i][j] < alloc[i][j]) {
                printf("Invalid input: Max cannot be less than Allocation.\\n");
                return 1;
            }
            need[i][j] = max[i][j] - alloc[i][j];
        }

    printf("Enter Available vector:\\n");
    for (int j = 0; j < m; j++) {
        scanf("%d", &avail[j]);
        work[j] = avail[j];
    }

    while (count < n) {
        int found = 0;
        for (int i = 0; i < n; i++) {
            if (!finish[i]) {
                int canFinish = 1;
                for (int j = 0; j < m; j++) {
                    if (need[i][j] > work[j]) {
                        canFinish = 0;
                        break;
                    }
                }
                if (canFinish) {
                    for (int j = 0; j < m; j++)
                        work[j] += alloc[i][j];
                    safe[count++] = i;
                    finish[i] = 1;
                    found = 1;
                }
            }
        }
        if (!found)
            break;
    }

    if (count == n) {
        printf("\\nSystem is in a SAFE state.\\nSafe sequence: ");
        for (int i = 0; i < n; i++) {
            printf("P%d", safe[i]);
            if (i < n - 1) printf(" -> ");
        }
        printf("\\n");
    } else {
        printf("\\nSystem is NOT in a safe state. No safe sequence found.\\n");
    }
    return 0;
```

```
  }
#include <stdio.h>

int main(void) {
int n, m;
int alloc[10][10], max[10][10], need[10][10];
int avail[10], work[10], finish[10] = {0};
int safe[10], count = 0;

printf("Enter number of processes (max 10): ");
scanf("%d", &n);
printf("Enter number of resource types (max 10): ");
scanf("%d", &m);

if (n < 1 || n > 10 || m < 1 || m > 10) {
printf("Invalid number of processes or resources.\\n");
return 1;
}

printf("Enter Allocation matrix:\\n");
for (int i = 0; i < n; i++)
for (int j = 0; j < m; j++)
scanf("%d", &alloc[i][j]);

printf("Enter Maximum matrix:\\n");
for (int i = 0; i < n; i++)
for (int j = 0; j < m; j++) {
scanf("%d", &max[i][j]);
if (max[i][j] < alloc[i][j]) {
printf("Invalid input: Max cannot be less than Allocation.\\n");
return 1;
}
need[i][j] = max[i][j] - alloc[i][j];
}

printf("Enter Available vector:\\n");
for (int j = 0; j < m; j++) {
scanf("%d", &avail[j]);
work[j] = avail[j];
}

while (count < n) {
int found = 0;

for (int i = 0; i < n; i++) {
if (!finish[i]) {
int canFinish = 1;
for (int j = 0; j < m; j++) {
if (need[i][j] > work[j]) {
canFinish = 0;
break;
}
}

if (canFinish) {
for (int j = 0; j < m; j++)
```

```
work[j] += alloc[i][j];

safe[count++] = i;
finish[i] = 1;
found = 1;
}
}
}

if (!found)
break;
}

if (count == n) {
printf("\\nSystem is in a SAFE state.\\nSafe sequence: ");
for (int i = 0; i < n; i++) {
printf("P%d", safe[i]);
if (i < n - 1) printf(" -> ");
}
printf("\\n");
} else {
printf("\\nSystem is NOT in a safe state. No safe sequence found.\\n");
}

return 0;
}
```

7. SAMPLE INPUT AND OUTPUT

Example input (3 processes, 3 resource types):

```
Enter number of processes (max 10): 3
Enter number of resource types (max 10): 3
Enter Allocation matrix:
1 0 1
1 1 0
0 1 1
Enter Maximum matrix:
2 1 1
1 2 1
1 1 2
Enter Available vector:
1 1 1
```

Expected output:

```
System is in a SAFE state.
Safe sequence: P0 -> P1 -> P2
```

8. TESTING

9. ADVANTAGES

- Helps avoid entering an unsafe resource-allocation state.
- Uses a clear and systematic method to check resource availability.
- Makes the concepts of Allocation, Maximum, Need and Available easy to understand.

• Can be implemented with basic C concepts such as arrays, loops and conditions.

10. LIMITATIONS

• The maximum resource requirement of each process must be known in advance.

• The basic implementation is limited to 10 processes and 10 resource types.

• The program demonstrates the safety check; it does not manage real operating-system resources.

• Repeated safety checks can add overhead in large systems.

11. APPLICATIONS

• Learning resource allocation and deadlock avoidance.

• Operating Systems laboratory demonstrations.

• Simulation of systems where multiple processes compete for limited resources.

12. CONCLUSION

The Banker's Algorithm provides a systematic way to determine whether a resource allocation can leave the system in a safe state. In this mini project, a C program calculates the Need matrix and searches for a safe sequence using the Available resources. The project demonstrates an important operating-system concept and improves understanding of arrays, loops, conditions and resource management. It is a learning simulation and is not intended to replace a real operating system's resource manager.

13. REFERENCES

• Abraham Silberschatz, Peter B. Galvin and Greg Gagne, Operating System Concepts.

• Operating Systems course notes and laboratory material.

• C programming language reference material for arrays, loops and input/output.

7. SAMPLE INPUT AND OUTPUT

Example input (3 processes, 3 resource types):

```
3
3
1 0 1
1 1 0
0 1 1
2 1 1
1 2 1
1 1 2
1 1 1
```

Expected output:

```
System is in a SAFE state.\nSafe sequence: P0 -> P1 -> P2
```

8. TESTING

Test case	Expected behaviour	Status
Safe input	At least one safe sequence is found.	Pass
Unsafe input	Reports that no safe sequence was found.	Pass
Invalid dimensions	Rejects values outside 1 to 10.	Pass
Invalid Max/Allocation	Reports invalid input if Max is less than Allocation.	Pass